



UNIDADE IV

Programação Orientada
a Objetos Com C#

Prof. MSc. Tarcísio Peres

Conteúdo da Unidade IV

Erros e testes

- Exceções, hierarquia, tratamento e criação de exceções
- Testes com xUnit e uso do Gerenciador de Testes

Memória e concorrência introdutória

- Valor × referência, GC, struct × class
- Task, async/await, cancelamento com CancellationToken

Erros em C#: três camadas

- O compilador detecta violações estáticas (regras e tipos).
- Em execução, falhas viram exceções (System.Exception e derivadas).
- Erros de lógica produzem resultados errados sem sinais técnicos.
- Robustez combina: compilador + exceções + testes + observabilidade.

Erros de compilação

- Surgem quando o código viola a especificação da linguagem.
- Exemplos: identificador inexistente, conversão impossível, sintaxe inválida.
- O compilador barra a geração do executável: falha detectada cedo.
- Servem como “primeira barreira” contra inconsistências.

Exceções em tempo de execução

- Aparecem quando o código compila, mas falha ao rodar.
- São objetos derivados de `System.Exception`.
- Exemplos comuns: `NullReferenceException`, `IndexOutOfRangeException`.
- Outros: `DivideByZeroException`; falhas de I/O como subclasses de `IOException`.

O que uma exceção carrega

- Message: descrição do problema.
- StackTrace: pilha de chamadas no ponto da falha.
- InnerException: preserva a causa original ao encapsular exceções.
- Essas informações orientam logs, diagnósticos e suporte.

Try / catch / finally

- try envolve o trecho que pode falhar.
- Ao lançar exceção, o runtime busca um catch compatível na pilha.
- catch trata: log, tradução para o domínio, ou repropagação (throw).
- finally executa sempre: liberação de recursos; when filtra cenários.

```
try
{
    // Código que pode lançar exceções
    int resultado = Numerador / Denominador;
    Arquivo.Write("Salvando dados...");
}
catch (DivideByZeroException ex)
{
    // Tratamento específico de divisão por zero
    Console.WriteLine($"Erro de divisão: {ex.Message}");
}
catch (IOException ex)
{
    // Tratamento de erros de E/S
    Console.WriteLine($"Erro de E/S: {ex.Message}");
}
catch (Exception ex)
{
    // Tratamento genérico de outras exceções
    Console.WriteLine($"Erro inesperado: {ex.Message}");
}
finally
{
    // Código sempre executado (por exemplo, liberar recursos)
    Console.WriteLine("Operação finalizada.");
}
```

Exceção vs. fluxo normal do domínio

- Casos previsíveis: entrada inválida, registro ausente, cancelamento.
- Modelar como fluxo normal costuma ser mais claro (if/switch + retorno).
- Padrão TryParse: retorna bool e evita exceções em condições comuns.
- Exceções “fortes” sinalizam quebra de contrato/invariante ou falha do ambiente.

Exemplo: divisão por zero

- Se o denominador vem do usuário, zero é cenário previsível → validar antes.
- `DivideByZeroException` indica que a cadeia não filtrou um valor inválido.
- Para o runtime, é erro de execução; para o design, faltou validação.
- Use exceções para violações, não para substituir checagens esperadas.

Erros de lógica

- Não geram erro do compilador nem Exception automaticamente.
- Exemplos: fórmula incorreta, fronteira ignorada, laço com item a mais/menos.
- Saídas podem parecer “normais”, mas violam requisitos do domínio.
- Defesa principal: testes bem desenhados, revisão de código e especificações claras.

Testes automatizados, CI e DevOps

- Testes automatizados são base de qualidade e do desenvolvimento ágil.
- Em CI/CD, rodar testes cedo ajuda a entregar software confiável.
- Testes capturam casos de fronteira que passam em testes manuais.
- Suíte reexecutada após mudanças atua como testes de regressão (pipeline).

xUnit e a estrutura de um teste

- xUnit é muito usado no ecossistema .NET moderno (unidade e integração).
- Testes são código de primeira classe, alinhados a práticas ágeis.
- Ciclo típico: setup/fixture → executar → verificar (assert) → teardown.
- Bug fix comum: reproduzir em teste, corrigir, manter teste na regressão.

Interatividade

Em qual cenário é mais apropriado evitar exceções e usar controle de fluxo (if/retorno)?

- a) Corrupção de dados em disco durante leitura.
- b) Perda súbita de conectividade em uma transação crítica.
- c) Usuário digitou um valor inválido em um formulário.
- d) `NullPointerException` em código interno que deveria garantir invariantes.
- e) Acesso fora dos limites de um array.

Resposta

Em qual cenário é mais apropriado evitar exceções e usar controle de fluxo (if/retorno)?

- a) Corrupção de dados em disco durante leitura.
- b) Perda súbita de conectividade em uma transação crítica.
- c) **Usuário digitou um valor inválido em um formulário.**
- d) `NullPointerException` em código interno que deveria garantir invariantes.
- e) Acesso fora dos limites de um array.

Testes com xUnit e Gerenciador de Testes

- xUnit.net é um dos frameworks de testes de unidade mais usados em C#.
- Criado pelos autores originais do NUnit, com foco “minimalista” (testes pequenos e legíveis).
- Prefere convenções (construtor/IDisposable) a atributos de setup/teardown.
- Integração nativa com o Visual Studio via Gerenciador de Testes.

Atributos: [Fact] e [Theory]

[Fact]

- Teste sem parâmetros; deve ser verdadeiro em qualquer execução.

[Theory]

- Teste parametrizado; valida o comportamento com diferentes entradas.
- Casos podem ser fornecidos com [InlineData].
- Cada conjunto de dados é executado e reportado como um teste distinto.

[Fact]

```
public void Somar_2e2_Retorna4()  
=> Assert.Equal(4, Calculadora.Somar(2,2));
```

[Theory]

```
[InlineData(3)] [InlineData(4)]  
public void EhImpar(int v) { /* ... */ }
```

TDD e execução dos testes

- TDD: escrever um teste que falha → implementar → refatorar.
- Execução no terminal: `dotnet test`.
- Execução no IDE: Teste > Gerenciador de Testes > Executar Todos.
- Saídas incluem descoberta, execução, total e falhas.

Projeto xUnit: pacotes essenciais

- Ao criar um projeto xUnit, o template já adiciona referências básicas.

Pacotes necessários para descoberta/execução no Visual Studio:

- Microsoft.NET.Test.Sdk
 - Integração com o pipeline de testes do .NET/VS.
- xunit.runner.visualstudio
 - Runner que habilita o Test Explorer.
- Sem esses pacotes, o Gerenciador de Testes não encontra os métodos de teste.

```
<ItemGroup>
<PackageReference Include="Microsoft.NET.Test.Sdk" />
<PackageReference Include="xunit" />
<PackageReference Include="xunit.runner.visualstudio" />
</ItemGroup>
```

O que é o “runner” no xUnit?

- Componente que descobre, executa e registra resultados dos testes.
- Procura métodos marcados com [Fact]/[Theory] no assembly de testes.
- Instancia classes de teste, executa métodos e captura exceções.
- Reporta quantos testes passaram, falharam ou foram ignorados.

Pipeline do runner



Setup/Tearardown: construtor e IDisposable

- xUnit cria uma nova instância da classe de teste antes de cada teste.
- Código no construtor funciona como setup automático.
- Implementar IDisposable permite executar cleanup em Dispose().
- Substitui o padrão clássico [SetUp]/[TearDown] de frameworks legados.

```
public class RecursoTests : IDisposable
{
    public RecursoTests() { /* setup */ }

    [Fact] public void Teste() { }

    public void Dispose() { /* cleanup */ }
}
```

Assertions mais comuns

- `Assert.Equal(expected, actual)`: compara valores/objetos.
- `Assert.True` / `Assert.False`: valida condições booleanas.
- `Assert.Throws<T>(…)`: verifica exceções esperadas.
- Falhas exibem mensagem e stack trace no Gerenciador de Testes.

```
Assert.Equal(4, resultado);
```

```
Assert.True(condicao);
```

```
Assert.Throws<Exception>( () => Metodo());
```

Exemplo: classe Perfil (TinderApp)

- Perfil representa um usuário (Nome, Idade, Interesses, Localização).
- Armazena curtidas e matches em listas internas.
- Curtir(Perfil): registra curtida e detecta match por reciprocidade.
- EhCompatívelCom: exige interesse em comum (Intersect) e mesma cidade.

```
public class Perfil {  
    public string Nome { get; set; }  
    public int Idade { get; set; }  
    public List<string> Interesses { get; set; }  
    public string Localizacao { get; set; }  
  
    private List<Perfil> perfisCurtidos = new List<Perfil>();  
    private List<Perfil> matches = new List<Perfil>();  
  
    public Perfil(string nome, int idade, List<string> interesses, string  
localizacao) {  
        Nome = nome;  
        Idade = idade;  
        Interesses = interesses;  
        Localizacao = localizacao;  
    }  
    // ...  
}
```

Testando a lógica de match com xUnit

- Crie a solução com um projeto principal e um projeto xUnit na mesma solução.
- No projeto de testes, adicione referência ao projeto principal.
- Escreva testes para: curtida mútua, curtida unilateral e compatibilidade.
- Os asserts validam regras de negócio automaticamente.

```
[Fact]
public void MatchMutuo()
{
    alice.Curtir(bob);
    Assert.True(bob.Curtir(alice));
}
```

Executando e interpretando resultados

- Compile a solução e abra o Gerenciador de Testes.
- Execute todos os testes e observe aprovados vs. falhas.
- Um teste “didático” pode falhar para demonstrar divergência de expectativa.
- Benefícios: detectar regressões cedo, documentar comportamento e facilitar refatorações.

Interatividade

Em xUnit, como normalmente fazemos setup/teardown?

- a) Com métodos estáticos [SetUp] e [TearDown].
- b) Com construtor da classe de teste e IDisposable.Dispose().
- c) Com atributos [BeforeEach] e [AfterEach].
- d) Apenas configurando o .csproj com pacotes NuGet.
- e) Diretamente no runner, sem código na classe de testes.

Resposta

Em xUnit, como normalmente fazemos setup/teardown?

- a) Com métodos estáticos [SetUp] e [TearDown].
- b) Com construtor da classe de teste e `IDisposable.Dispose()`.
- c) Com atributos [BeforeEach] e [AfterEach].
- d) Apenas configurando o .csproj com pacotes NuGet.
- e) Diretamente no runner, sem código na classe de testes.

Memória e concorrência em C# (visão geral)

- Ambiente gerenciado ajuda, mas não elimina decisões de memória e paralelismo.
- Em aplicações grandes, surgem pausas, consumo excessivo e travamentos difíceis.
- Entender o runtime (objetos, GC, threads, tasks) é requisito de previsibilidade.
- Projeto de tipos (struct × class) impacta desempenho e legibilidade.

O mito do “GC resolve tudo”

- O GC previne vazamentos clássicos, mas pode introduzir pausas perceptíveis.
- Alocações excessivas aumentam a pressão sobre o coletor e elevam a latência.
- Abstrações de alto nível não dispensam medir alocação, cópia e retenção.
- Responsividade depende de escolhas de dados e do modelo de concorrência.

Projeto de tipos: struct × class

- struct: tipo por valor; atribuição copia os dados; ideal para dados pequenos.
- class: tipo por referência; identidade compartilhada; suporte à herança.
- A escolha afeta alocação (stack/inline/heap), custo de cópias e boxing.
- Também influencia invariantes e o raciocínio sobre “posse” de estado.

Como o CLR aloca objetos (new)

- Instâncias de class são alocadas no heap gerenciado e rastreadas pelo GC.
- Tamanho = campos do objeto + overhead (tipo e sincronização, por exemplo).
- Com espaço disponível: aloca, zera memória, invoca construtor.
- Um ponteiro interno avança para o próximo bloco livre (alocação sequencial).

Por que alocar é rápido

- Alocação linear no heap é barata e favorece localidade de referência.
- Objetos relacionados tendem a ficar próximos → menos faltas de cache.
- O custo aparece quando a memória cresce e o GC precisa intervir.
- Otimização prática: reduzir alocações e a vida útil de objetos.

GC do .NET: rastreamento e compactação

- O GC é por rastreamento (não por contagem de referências).
- Durante a coleta: suspende threads, limpa marcas, marca a partir das raízes.
- Objetos não marcados são considerados inalcançáveis e podem ser removidos.
- Compactação move sobreviventes para manter espaço contíguo e localidade.

Valor × referência: semântica e exemplos

- Tipos por valor armazenam a instância; atribuição cria cópia independente.
- Tipos por referência armazenam um ponteiro; atribuição copia a referência.
- Dois nomes podem referir o mesmo objeto; mutações são compartilhadas.
- Isso influencia APIs, coleções e o custo de passagem de parâmetros.

Structs na prática: benefícios e armadilhas

- Use structs para tipos pequenos, de semântica de valor e preferencialmente imutáveis.
- Benefício: menos alocações no heap e menor trabalho do GC.
- Armadilha: structs grandes aumentam custo de cópia e podem piorar desempenho.
- Evite struct para entidades com identidade e estado mutável complexo.

Quando um struct vai parar no heap

- Boxing: ao tratar struct como object/interface, cria-se uma “caixa” no heap.
- Arrays são objetos no heap; elementos struct ficam inline no array.
- Como campo de class: o objeto dono está no heap; o struct fica inline nele.
- Esses detalhes afetam coleções genéricas e interoperabilidade.

Concorrência moderna: threads → tasks → async

- Threads são escalonadas pelo SO; uso direto expõe riscos e complexidade.
- Problemas típicos: condição de corrida, deadlock e uso inadequado de recursos.
- Tasks elevam o nível: foco na unidade de trabalho, não na thread individual.
- async/await ajuda em I/O-bound; CancellationToken viabiliza cancelamento cooperativo.

Interatividade

Qual afirmação descreve corretamente um aspecto do comportamento do GC no .NET?

- a) O GC do .NET usa contagem de referências e libera objetos assim que o último ponteiro é removido.
- b) O GC do .NET é um coletor por rastreamento, que interrompe as threads, marca objetos alcançáveis a partir das raízes e remove os não marcados.
- c) O CLR aloca objetos no heap, mas não zera a memória antes de invocar o construtor.
- d) Variáveis de tipos por valor são consideradas raízes do GC, pois podem apontar para objetos no heap.
- e) A compactação do heap aumenta a fragmentação para evitar a movimentação de objetos.

Resposta

Qual afirmação descreve corretamente um aspecto do comportamento do GC no .NET?

- a) O GC do .NET usa contagem de referências e libera objetos assim que o último ponteiro é removido.
- b) O GC do .NET é um coletor por rastreamento, que interrompe as threads, marca objetos alcançáveis a partir das raízes e remove os não marcados.
- c) O CLR aloca objetos no heap, mas não zera a memória antes de invocar o construtor.
- d) Variáveis de tipos por valor são consideradas raízes do GC, pois podem apontar para objetos no heap.
- e) A compactação do heap aumenta a fragmentação para evitar a movimentação de objetos.

Tasks e programação concorrente em C#

- Task (tarefa) = operação assíncrona de alto nível, potencialmente paralela.
- TPL distribui tarefas pelo Thread Pool, sem gerenciar threads manualmente.
- Abstrai criação/sincronização de threads → código mais seguro e legível.

Processo vs. Thread (conceitos-chave)

- Processo: contêiner pesado (memória, handles, conexões e recursos globais).
- Thread: linha de execução que consome CPU dentro do processo (pilha e registradores próprios).
- Threads compartilham o mesmo espaço de endereçamento do processo; cada uma tem seu fluxo.

Threads no .NET e o Thread Pool

- CLR expõe threads gerenciadas (`System.Threading.Thread`), geralmente mapeadas ao SO.
- Estilo moderno em C#: preferir TPL/Task e deixar o runtime gerenciar threads.
- Thread Pool mantém worker threads reutilizáveis e distribui as unidades de trabalho.

Task: unidade de trabalho da TPL

- Representa trabalho em andamento, concluído, cancelado ou com falha.
- Suporta composição: continuations, Task.WhenAll e Task.WhenAny.
- Exemplo: Task.Run(() => /* trabalho */) e depois await para aguardar.

```
async Task<int> SomaAsync(int a, int b) {  
    // Executa o cálculo de soma numa tarefa paralela sem  
    bloquear o thread atual  
    return await Task.Run(() => a + b);  
}
```

TAP + `async/await`

- Task-Based Asynchronous Pattern: APIs assíncronas retornam `Task` ou `Task<T>`.
- `async/await` permite escrever fluxo “sequencial” sem bloquear a thread chamadora.
- Resultado: UI responsiva e melhor escalabilidade em serviços.

Como async/await funciona por baixo

- Métodos async que retornam Task/Task<T> viram uma máquina de estados.
- Em await de Task não concluída, o método registra continuação e retorna controle.
- Quando a Task completa, a continuação é reagendada e o fluxo retoma no await.

Exemplo: soma assíncrona (Task.Run)

```
async Task<int> SomaAsync(int a, int b) {  
    return await Task.Run(() => a + b);  
}  
await SomaAsync(3, 4) //libera o chamador e retoma com o resultado 7
```

I/O-bound

- I/O-bound = tempo dominado por espera (rede, disco, banco, soquetes), não por CPU.
- Chamada síncrona bloqueia a thread; chamada assíncrona libera para outras tarefas.
- Ex.: `HttpClient.GetStringAsync(uri)` retorna `Task<string>` para uso com `await`.

Paralelismo com WhenAll/WhenAny e exceções

- `Task.Delay(ms)` espera de forma assíncrona (sem bloquear a thread).
- `Task.WhenAll` aguarda várias tarefas em paralelo e retorna quando todas terminam.
- Exceções em `Tasks` se propagam quando aguardadas (`await/WaitAll`) e usam `try/catch`.

Cancelamento cooperativo com CancellationToken

- Chamador cria CancellationTokenSource e passa cts.Token aos métodos.
- Operação verifica ct.IsCancellationRequested ou usa APIs que aceitam o token
- Sinalização: cts.Cancel(); interrupção limpa: ct.ThrowIfCancellationRequested()
- Tokens podem ser combinados via CancellationTokenSource.CreateLinkedTokenSource

```
public async Task<int>
ObterContagemPalavrasAsync(CancellationToken ct) {
    // Simula delay para permitir cancelamento
    await Task.Delay(1000, ct);
    // Exemplo de chamada externa suportando cancelamento
    HttpResponseMessage resp = await
httpClient.GetAsync(urlArquivo, ct);
    string texto = await resp.Content.ReadAsStringAsync(ct);
    string[] palavras = texto.Split(new[]{' ', '\n', '\r'},
StringSplitOptions.RemoveEmptyEntries);
    ct.ThrowIfCancellationRequested(); // verifica cancelamento
antes de retornar
    return palavras.Length;
}
```

Interatividade

Qual afirmação descreve melhor a relação entre Task, threads e a Task Parallel Library (TPL) no modelo de concorrência moderno do C#?

- a) A Task é uma thread do sistema operacional; por isso, cada Task.Run cria sempre uma nova thread dedicada.
- b) A TPL substitui o Thread Pool; assim, as Tasks são executadas sem o uso de threads.
- c) A Task é uma abstração de alto nível de “unidade de trabalho”, normalmente executada sobre threads reaproveitadas do Thread Pool, enquanto o desenvolvedor trabalha mais com Tasks do que com Thread diretamente.
- d) O await bloqueia o thread chamador até a Task terminar, garantindo que o fluxo não continue antes do resultado.
- e) O cancelamento com CancellationToken interrompe imediatamente qualquer Task por força do runtime, sem depender de o código cooperar.

Resposta

Qual afirmação descreve melhor a relação entre Task, threads e a Task Parallel Library (TPL) no modelo de concorrência moderno do C#?

- a) A Task é uma thread do sistema operacional; por isso, cada Task.Run cria sempre uma nova thread dedicada.
- b) A TPL substitui o Thread Pool; assim, as Tasks são executadas sem o uso de threads.
- c) A Task é uma abstração de alto nível de “unidade de trabalho”, normalmente executada sobre threads reaproveitadas do Thread Pool, enquanto o desenvolvedor trabalha mais com Tasks do que com Thread diretamente.
- d) O await bloqueia o thread chamador até a Task terminar, garantindo que o fluxo não continue antes do resultado.
- e) O cancelamento com CancellationToken interrompe imediatamente qualquer Task por força do runtime, sem depender de o código cooperar.

Referências

- ALBAHARI, J.; ALBAHARI, B. *LINQ pocket reference*. 1. ed. Sebastopol, CA: O'Reilly Media, 2008.
- ALBAHARI, J. *C# 10 in a nutshell: the definitive reference*. 1. ed. Sebastopol, CA: O'Reilly Media, 2022.
- ALLS, J. *Clean code with C#*. 2. ed. Birmingham: Packt Publishing, 2023.
- ARAÚJO, E. C. *Orientação a objetos em C#: conceitos e implementações em .NET*. São Paulo: Casa do Código, 2017.
- BLOCH, J. *Effective Java*. 3. ed. Boston: Addison-Wesley, 2018.
- BOOCH, G. *et al. Object-oriented analysis and design with applications*. 3. ed. Boston: Addison-Wesley, 2007.
- CHACON, S.; STRAUB, B. *Pro Git*. 2. ed. Berkeley: Apress, 2014.
- CLEARY, S. *Concurrency in C# cookbook: asynchronous, parallel, and multithreaded programming*. 2. ed. Sebastopol, CA: O'Reilly Media, 2019.

Referências

- CLYMER, A.; BLEWETT, R. *Pro asynchronous programming with .NET*. New York: Apress, 2013.
- DEITEL, P.; DEITEL, H. *Visual C#: how to program*. 6. ed. Boston: Pearson, 2016.
- ECMA INTERNATIONAL. *ECMA-334*. Geneva: ECMA International, 2023. Disponível em: <https://tinyurl.com/3mya2btj>. Acesso em: 27 nov. 2025.
- EVANS, E. *Domain-driven design: atacando as complexidades no coração do software*. 3. ed. Rio de Janeiro: Alta Books, 2015.
- FORSGREN, N.; HUMBLE, J.; KIM, G. *Accelerate: the science of lean software and DevOps – building and scaling high performing technology organizations*. Portland, OR: IT Revolution, 2018.
- FOWLER, M. *Refatoração: aperfeiçoando o design de códigos existentes*. 2. ed. Porto Alegre: Novatec, 2020.
- FREEMAN, A.; RATTZ, J. C. *Pro LINQ: language integrated query in C# 2010*. 1. ed. Berkeley: Apress, 2010.

Referências

- GADDIS, T. *Starting out with visual C#*. 4. ed. São Paulo: Pearson, 2017.
- GAMMA, E. *et al. Design patterns: elements of reusable object-oriented software*. 1. ed. Reading: Addison-Wesley, 1994.
- GAMMA, E. *et al. Padrões de projeto: soluções reutilizáveis de software orientado a objetos*. Tradução de Luiz A. Meirelles Salgado. Porto Alegre: Bookman, 2005.
- GRIFFITHS, I. *Programming C# 10: build cloud, web, and desktop applications*. 1. ed. Sebastopol, CA: O'Reilly Media, 2022.
- HALL, G. M. *Adaptive code via C#: agile coding with design patterns and SOLID principles*. 2. ed. Redmond: Microsoft Press, 2017.
- HORSTMANN, C. S. *Object-oriented design & patterns*. 2. ed. Hoboken: John Wiley & Sons, 2006.
- HUNT, D.; THOMAS, A. *O programador pragmático: de aprendiz a mestre*. Porto Alegre: Bookman, 2010.
- KHORIKOV, V.; OSHEROVE, R. *The art of Unit testing*. 3. ed. Shelter Island: Manning, 2024.

Referências

- KIM, G. *et al. The DevOps handbook: how to create world-class agility, reliability, & security in technology organizations*. 2. ed. Portland, OR: IT Revolution, 2021.
- LARMAN, C. *Utilizando UML e padrões: uma introdução à análise e ao projeto orientados a objetos e ao desenvolvimento iterativo*. 3. ed. Porto Alegre: Bookman, 2007.
- LIBERTY, J. *Programming C#*. 4. ed. Sebastopol, CA: O'Reilly Media, 2005.
- LOELIGER, J.; McCULLOUGH, M. *Version control with Git: powerful tools and techniques for collaborative software development*. 2. ed. Sebastopol, CA: O'Reilly Media, 2012.
- MARTIN, R. C. *Arquitetura limpa: o guia do artesão para estrutura e design de software*. Rio de Janeiro: Alta Books, 2019.
- MARTIN, R. C. *Código limpo: habilidades práticas do Agile Software*. Rio de Janeiro: Alta Books, 2009.
- MARTIN, R. C.; MARTIN, M. *Princípios, padrões e práticas ágeis em C#*. Tradução de João Eduardo Nóbrega Tortello. Porto Alegre: Bookman, 2011.
- MESZAROS, G. *xUnit test patterns: refactoring test code*. Boston: Addison-Wesley, 2007.

Referências

- MICHAELIS, M. *Essential C# 12.0*. 8. ed. Boston: Addison-Wesley, 2023.
- MICROSOFT. *Comparar as edições do Visual Studio*. [s.d.]. Disponível em: <https://tinyurl.com/ym7j8c8d>. Acesso em: 26 nov. 2025.
- PAINTER, S. J. *Functional programming with C#*: create more supportable, robust, and testable code. Sebastopol, CA: O'Reilly Media, 2023.
- PRICE, M. J. *C# 14 and .NET 10*: modern cross-platform development fundamentals – build modern websites and services with ASP.NET Core, Blazor, and EF Core using Visual Studio 2026. 10. ed. Birmingham: Packt Publishing, 2025.
- SCHILDT, H. *C# 4.0*: the complete reference. New York: McGraw-Hill, 2010.
- SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. *Operating system concepts*. 10. ed. Hoboken, NJ: John Wiley & Sons, 2018.
- SKEET, J. *C# in depth*. 4. ed. Shelter Island: Manning Publications, 2019.

Referências

- STACK OVERFLOW. *2025 Stack Overflow developer survey*. 2025. Disponível em: <https://tinyurl.com/32tv4y3t>. Acesso em: 21 nov. 2025.
- TROELSEN, A.; JAPIKSE, P. *Pro C# 7: with .NET and .NET Core*. 8. ed. New York: Apress, 2017.
- TROELSEN, A.; JAPIKSE, P. *Pro C# 9 with .NET 5: foundational principles and practices in programming*. 10. ed. New York: Apress, 2021.
- TROELSEN, A.; JAPIKSE, P. *Pro C# 10 with .NET 6: foundational principles and practices in programming*. 11. ed. New York: Apress, 2022.
- WAGNER, B. *Effective C#: 50 specific ways to improve your C#*. Boston: Addison-Wesley, 2004.
- WEISFELD, M. *The object-oriented thought process*. 4. ed. Boston: Addison-Wesley, 2013.
- WOOLEY, J.; EICHERT, S.; MARGUERIE, F. *LINQ em ação*. 1. ed. Rio de Janeiro: Ciência Moderna, 2009.

ATÉ A PRÓXIMA!